P0330R7
Literal Suffixes for ptrdiff_t and size_t

Published Proposal, 2019-08-08

Authors:
JeanHeyd Meneide
Rein Halbersma

Audience:
CWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:
C++23

Latest:
https://thephd.github.io/vendor/future_cxx/papers/d0330.html

Abstract

This paper proposes core language suffixes for size_t and ptrdiff_t and their associated types.

Table of Contents

1. Revision History

1.1. Revision 7 - June 17th, 2019

- Added additional motivating examples (thanks David Stone).
- Fixed derp in wording (thanks Roger Orr).
- Evolution Working Group evaluated the paper, passing the `uz`/`z` literals, while denying the `ut`/`t` literals.
- Accept the uz/z literal suffixes for integers as in P0330 for C++23?

```
SF  F  N  A SA
 4 19 12  2  0
```

- Accept the ut/t literal suffixes for integers as in P0330 for C++23?

```
SF  F  N  A SA
 0  8 15 10  2
```

- Going to Core Working Group for C++23!

1.2. Revision 6 - June 17th, 2019

- Added additional motivating examples (thanks Alberto Escrig).
- Update wording to reflect against [n4820].

1.3. Revision 5 - February 8th, 2019

- Added additional motivation regarding template parameters.
- Add additional feature test macros for each type of suffix.

1.4. Revision 4 - January 21st, 2019

- Discussed additional reasoning due to `<span>` changes with `ssize(const T&)` and others.
- Wording is now relative to [n4778], the latest C++ Standard working draft.

1.5. Revision 3 - November 26th, 2018

- Strengthened rationale for the current set of suffixes in §4.1 Using t for ptrdiff_t and zu for size_t?.

1.6. Revision 2 - October 1st, 2018

- Published as P0330R2; change reply-to and point of contact from Rein Halbersma to JeanHeyd Meneide, who revitalized paper according to feedback from Rein Halbersma, and all of LWG. Overwhelming consensus for it to be a Language Feature instead, proposal rewritten as Language proposal with wording against [n4762].

1.7. Revision 1 - October 12th, 2017

- Published as P0330R1; expanded the survey of existing literals. Synced the proposed wording with the Working Draft WG21/N4687. Moved the reference implementation from BitBucket to GitHub.

1.8. Revision 0 - November 21st, 2014

- Initial release; published as N4254.
- Published as P0330R0; summarized LEWG's view re N4254; dropped the proposed suffix for ptrdiff_t; changed the proposed suffix for size_t to zu; added survey of existing literal suffixes.

2. Feedback on Revisions

Polls are in the form **Strongly in Favor** | **Favor** | **Neutral** | **Against** | **Strongly Against**. The polls on Revision 1 were as follows, from an EWG meeting with joint LWG/EWG participation at the WG21 2017 Albuquerque meeting.

Proposal as presented, i.e., are we OK with the library solution going forward?

0 | **6** | **5** | **7** | **4**

We translated this as strong discouragement to pursue this feature as a set of user-defined literals. A second poll was taken.

Do we want to solve this problem with a language feature?

2 | **15** | **0** | **2** | **2**

We considered this overwhelming consensus for it to be a language feature instead, culminating in this paper after much feedback.

3. Motivation

<table>
<tr><th>Currently</th><th>With Proposal</th></tr>
<tr><td>

```
std::vector<int> v{0, 1, 2, 3};
for (auto i = 0u, s = v.size(); i < s; ++i) {
        /* use both i and v[i] */
}
```

⚠ - Compiles on 32-bit, truncates (maybe with warnings) on 64-bit

```
std::vector<int> v{0, 1, 2, 3};
for (auto i = 0, s = v.size(); i < s; ++i) {
        /* use both i and v[i] */
}
```

✖ - Compilation error

</td><td>

```
std::vector<int> v{0, 1, 2, 3};
for (auto i = 0uz, s = v.size(); i < s; ++i) {
        /* use both i and v[i] */
}
```

✔ - Compiles with no warnings on 32-bit or 64-bit

</td></tr>
<tr><td>

```
auto it = std::find(boost::counting_iterator(0),
        boost::counting_iterator(v.size()), 3);
```

</td><td>

```
auto it = std::find(boost::counting_iterator(0uz),
        boost::counting_iterator(v.size()), 3uz);
```

</td></tr>
</table>

✖ - Compilation error	✔ - Compiles with no warnings on 32-bit or 64-bit

`std::size_t space_param = /* ... */;`
`std::size_t clamped_space = std::max(0,`
`  std::min(54, space_param)`
`);`
`vec.reserve(clamped_space);`
	`std::size_t space_param = /* ... */;`
	`std::size_t clamped_space = std::max(0uz,`
	`  std::min(54uz, space_param)`
	`);`
	`vec.reserve(clamped_space);`

`std::span<int> s = /* init */;`
`std::ptrdiff_t clamped_space = std::max(0,`
`  std::min(24, std::ssize(s))`
`);`
	`std::span<int> s = /* init */;`
	`std::ptrdiff_t clamped_space = std::max(0t,`
	`  std::min(24t, std::ssize(s))`
	`);`

✖ - Compilation error; OR, ⚠ - Compiles, but becomes excessively verbose with `static_cast` or `(type)` casts	✔ - Compiles with no warnings on 32-bit or 64-bit

`template <class... TYPES>`
`constexpr`
`void`
`tuple<TYPES...>::swap(tuple& other)`
`noexcept((is_nothrow_swappable_v<TYPES> and ...))`
`{`
`  for...(constexpr size_t N : view::iota(0, sizeof...(TYPES))) {`
`  swap(get<N>(*this), get<N>(other));`
`  }`
`}`
	`template <class... TYPES>`
	`constexpr`
	`void`
	`tuple<TYPES...>::swap(tuple& other)`
	`noexcept((is_nothrow_swappable_v<TYPES> and ...))`
	`{`
	`  for...(constexpr size_t N : view::iota(0uz,`
	`  swap(get<N>(*this), get<N>(other));`
	`  }`
	`}`

✖ - Compilation error; OR, ⚠ - Compiles, but becomes excessively verbose with `static_cast` or `(type)` casts	✔ - Compiles with no warnings on 32-bit or 64-bit

Consider this very simple code to print an index and its value:

```
std::vector<int> v{0, 1, 2, 3};
for (auto i = 0; i < v.size(); ++i) {
        std::cout << i << ": " << v[i] << '\n';
}
```

This code can lead to the following warnings:

```
main.cpp: In function 'int main()':
main.cpp:warning: comparison of integer expressions
of different signedness: 'int' and 'long unsigned int' [-Wsign-compare]
    for (auto i = 0; i < v.size(); ++i) {
                     ~~^~~~~~~~~~
```

It grows worse if a user wants to cache the size rather than query it per-iteration:

```
std::vector<int> v{0, 1, 2, 3};
for (auto i = 0, s = v.size(); i < s; ++i) {
        /* use both i and v[i] */
}
```

Resulting in a hard compiler error:

```
main.cpp: In function 'int main()':
main.cpp:8:10: error: inconsistent deduction
for 'auto': 'int' and then 'long unsigned int'
     for (auto i = 0, s = v.size(); i < s; ++i) {
          ^~~~
```

This paper proposes adding a `zu`s literal suffix that creates `size_t` literals, making the following warning-free:

```
for (auto i = 0zu; i < v.size(); ++i) {
        std::cout << i << ": " << v[i] << '\n';
}
```

It also makes this code compile without `error: no matching function for call to 'min(int, std::vector<int>::size_type)'` and similar:

```
#include <algorithm>
#include <vector>

int main() {
        std::vector<int> v;
        /* work with v... */

        std::size_t clamped_space = std::max(0zu,
                std::min(54zu, v.size()) // error without suffix
        );

        return 0;
}
```

More generally:

- int is the default type deduced from integer literals without suffix;

- comparisons, signs or conversion ranks with integers can lead to surprising results;
- `size_t` -- and now more frequently, `ptrdiff_t` because of `ssize()` -- are nearly impossible to avoid in the standard library for element access or `.size()` members;
- programmer intent and stability is hard to communicate portably with the current set of literals;
- surprises range from (pedantic) compiler errors to undefined behavior;
- existing integer suffixes (such as `ul`) are not a general solution, e.g. when switching compilation between 32-bit and 64-bit on common architectures;
- template parameters for multiple arguments often clash with what literals have;
- and, C-style casts and static_casts are verbose.

4. Design

Following the feedback from §2 Feedback on Revisions, we have dropped the `std::support_literals` User-Defined Literals and chose a Core Language Literal Suffix. We opine that it would better serve the needs of addressing the motivation.

As a language feature, the design of the suffixes becomes much simpler. The core language only has one format for its integer literal suffixes: the letter(s), with an optional `u` on either side of the letter(s) to make it unsigned, with the signed variant being the default on most architectures. We did not want to use `s` because `s` might mean `short` to some and there are people working on the `short float` paper currently wherein a suffix such as `sf` might surface. In this case, it would make some small amount of sense for the suffix `s` to also work for shorts, albeit that might have unforeseen consequences with standard-defined library literals.

The literal suffixes `z` and `uz`/`zu` alongside `t` and `ut`/`tu` were chosen to represent signed/unsigned `size_t` and `ptrdiff_t`, respectively. `decltype(0t)` will yield `ptrdiff_t` and `decltype(0uz)`/`decltype(0zu)` will yield `size_t`. Like other case-insensitive language literal suffixes, it will accept both `Z`/`T` and `z`/`t` (and `U` and `u` alongside of it). This follows the current convention of the core language to be able to place `u` and `z`/`t` in any order / any case for the suffix.

4.1. Using `t` for `ptrdiff_t` and `zu` for `size_t`?

Previous invocations of this paper used only `z` and `uz`/`zu`, mostly because there was no named type that represented what a signed `std::size_t` or an unsigned `std::ptrdiff_t` was. This made it awkward to place into the C++ wording for the author writing this paper. However, Core Wording experts (thanks Hubert Tong and Jens Maurer!) have helped elucidate that while the type may not have a formal name or type alias in the language, it is perfectly valid to say "the unsigned/signed integer type corresponding to {X}".

4.2. Why bother making a suffix for ptrdiff_t?

With the inclusion of a `ssize()` free function coming to the standard, this paper advocates for keeping a literal for `ptrdiff_t`. As the paper was going through the Library group earlier, `span`'s design decisions were not coming to a head and thusly the dialogue did not bring this up. With `span` now headed into C++20 and `ssize()` with it, having a modifier for `ptrdiff_t` is useful for consistency and helpful for success in a world where developers employ a lot of `auto` and `decltype`.

4.3. But what about {insert favorite suffix here}?

We designed the suffixes based on feedback from both EWG and Core members in the 3 mailing list posts corresponding to that discussion. We will take additional polls on the actual suffix desired by the Community before EWG.

For example, it was made clear during discussion that while some people would not lose any sleep over a suffix scheme such as `z` for `ptrdiff_t` and `uz`/`zu` for `size_t`, others were concerned that architectures (such as armv7-apple-darwin) produced answers such as `decltype(0zu) = unsigned long` for `size_t` and `decltype(0z) = int` for `ptrdiff_t`. They have the same range exponent on this architecture but the type disconnect would likely bother some folks. The current scheme is to avoid such a pairing of incongruent types.

4.4. What about the fixed/least/max (unsigned) int types?

This paper does not propose suffixes for the fixed size, at-least size, and max size integral types in the standard library or the language. This paper is focusing exclusively on `ptrdiff_t` and `size_t`. We have also been made aware of another paper which may handle this separately and considers all the design space necessary for such. We feel it would be best left to LEWG to handle such a paper, since they are closer to library types.

5. Impact on the Standard

This feature is purely an extension of the language and has, to the best of our knowledge, no conflict with existing or currently proposed features. `z` and `t` are currently not a literal suffix in the language. As a proof of concept, it has a patch in GCC already according to this paper by Ed Smith-Rowland.

6. Proposed wording and Feature Test Macros

The following wording is relative to [n4820].

6.1. Proposed Feature Test Macro

The recommended feature test macro is `__cpp_size_t_suffix`.

6.2. Intent

The intent of this paper is to propose 2 language suffixes for integral literals of specific types. One is for `ptrdiff_t`, one is for `size_t`. We follow the conventions set out for other literals in the standard. We define the suffix to produce types `size_t` and `ptrdiff_t` similar to how §5.13.7 Pointer Literals [lex.nullptr] introduces `std::nullptr_t`.

6.3. Proposed Wording

Modify §5.13.2 Integer Literals [**lex.icon**] with additional suffixes:

> ***integer-suffix***:
> *unsigned-suffix long-suffix*$_{opt}$
> *unsigned-suffix long-long-suffix*$_{opt}$
> *unsigned-suffix size-suffix*$_{opt}$
> *long-suffix unsigned-suffix*$_{opt}$
> *long-long-suffix unsigned-suffix*$_{opt}$
> *size-suffix unsigned-suffix*$_{opt}$
>
> *unsigned-suffix*: one of
> u U
>
> *long-suffix*: one of
> l L
>
> *long-long-suffix*: one of
> ll LL
>
> *size-suffix*: one of
> z Z

Append to §5.13.2 Integer Literals [**lex.icon**]'s **Table 7** four additional entries:

> | Suffix | Decimal literal | Binary, octal, or hexadecimal literal |
> |---|---|---|
> | z or Z | the signed integer type corresponding to `size_t` | the signed integer type corresponding to `size_t` |
> | Both u or U and z or Z | `size_t` | `size_t` |

Append to §14.8 Predefined macro names [**cpp.predefined**]'s **Table 17** with one additional entry:

> | Macro name | Value |
> |---|---|
> | `__cpp_size_t_suffix` | 201902L |

7. Acknowledgements

Thank you to Rein Halbersma, who started this paper and put in the necessary work for r0 and r1. Thank you to Walter E. Brown, who acted as *locum* on this paper before the Committee twice and gave us valuable feedback on wording. Thank you to Lounge<C++>'s Cicada for encouraging us to write this paper. Thank you to Hubert Tong and Jens Maurer for giving us a few pointers on where in the Core Language to modify things for such a paper and what words to use. Thank you to Tim Song for wording advice.

We appreciate your guidance as we learn to be a better Committee member and represent the C++ community's needs more more efficiently and effectively in the coming months.

References

Informative References

[GCC-IMPLEMENTATION]
Ed Smith-Rowland. [[C++ PATCH]] Implement C++2a P0330R2 - Literal Suffixes for ptrdiff_t and size_t. October 21st, 2018. URL: https://gcc.gnu.org/ml/gcc-patches/2018-10/msg01278.html

[N4762]
ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. N4762- Working Draft, Standard for Programming Language C++. May 10th, 2018. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf

[N4778]
ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. N4778 - Working Draft, Standard for Programming Language C++. November 26th, 2018. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4778.pdf

[N4820]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. N4820 - Working Draft, Standard for Programming Language C++. January 21st, 2019. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4800.pdf